

Guia Rápido de Git para Projetos Colaborativos com Exemplo Prático

Este guia é uma referência melhorada e completa para usares o Git em projetos colaborativos. Inclui passos essenciais, comandos úteis, boas práticas e um exemplo prático baseado num use case simples de um projeto web (HTML, CSS e JS). É ideal como "cábula" para consulta futura.

1 Configuração Inicial

Antes de começar, verifica estes pré-requisitos:

- **Instala o Git:** Baixa em <https://git-scm.com/>. No Mac: `brew install git` (com Homebrew). No Windows/WSL: usa o instalador ou `sudo apt install git`.
- **Instala o VS Code:** Baixa em <https://code.visualstudio.com/>. Instala a extensão "GitHub Pull Requests and Issues" e "Live Server" para testes.
- **Configura o Git:** Define o teu nome e email (faz uma vez):

```
git config --global user.name "Teu Nome"
git config --global user.email "teu.email@exemplo.com"
```

- **Cria uma conta no GitHub:** Vai a <https://github.com/>.

2 Criando e Clonando um Repositório

- **Cria um repositório no GitHub:**
 1. Inicia sessão no GitHub.
 2. Clica em "New" e dá um nome (ex: "projeto-web-simples").
 3. Adiciona um README.md inicial e cria.
- **Convida colaboradores:** Em "Settings» "Collaborators", adiciona usernames.
- **Clona localmente:**

```
git clone <URL-do-repositório>
cd <nome-da-pasta>
git status % Verifica: deve mostrar "On branch main...
            nothing to commit"
```

3 Fluxo de Trabalho com Branches

- Nunca trabalhes na main! Usa branches para cada funcionalidade.
- Cria uma branch:

```
git checkout -b feature/nome-da-funcionalidade
```

- Edita ficheiros no VS Code.
- Adiciona e commita:

```
git add <ficheiro> % Ou git add . para todos
git commit -m "Mensagem clara do que foi feito"
```

- Envia para o GitHub:

```
git push origin feature/nome-da-funcionalidade
```

- Atualiza antes de começar: Sempre faz git pull origin main.

4 Exemplo Prático: Use Case de Projeto Web Simples

Vamos simular um projeto colaborativo básico: uma página web com HTML, CSS e JS. "Ela" cria o repositório e convida "Tu". Ambos trabalham em branches separadas.

4.1 Passo 1: Criação e Clonagem

- Ela cria o repositório "projeto-web-simples" no GitHub com README.md e convida-te.
- Ambos clonam:

```
git clone https://github.com/username/projeto-web-simples.git
cd projeto-web-simples
```

Estrutura inicial:

```
projeto-web-simples/
  README.md
```

4.2 Passo 2: Branch para HTML (Ela faz)

```
git checkout -b feature/html-basico
```

- Cria index.html:

```
<!DOCTYPE html>
<html lang="pt">
<head>
  <meta charset="UTF-8">
  <title>Página Simples</title>
  <link rel="stylesheet" href="styles.css">
  <script src="script.js"></script>
</head>
```

```
<body>
  <h1>Olá, Mundo!</h1>
  <p>Esta é uma página web básica.</p>
</body>
</html>
```

- Commit e push:

```
git add index.html
git commit -m "Adiciona HTML básico da página"
git push origin feature/html-basico
```

Estrutura agora (na branch feature/html-basico):

```
projeto-web-simples/
  README.md
  index.html
```

4.3 Passo 3: Branches para CSS (Ela) e JS (Tu)

- Ela cria issue no GitHub para tarefas. - Ela para CSS:

```
git checkout main
git pull origin main
git checkout -b feature/css-estilos
```

- Cria styles.css:

```
body {
  background-color: lightblue;
  font-family: Arial, sans-serif;
}
h1 {
  color: navy;
}
```

- Commit e push. - Tu para JS (similar):

```
git checkout main
git pull origin main
git checkout -b feature/js-funcionalidade
```

- Cria script.js:

```
document.addEventListener('DOMContentLoaded', () => {
  alert('Página carregada!');
});
```

- Commit e push. Estrutura após pushes (por branch): - main: só README.md - feature/html-basico: + index.html - feature/css-estilos: + styles.css - feature/js-funcionalidade: + script.js

4.4 Passo 4: Pull Requests e Merge (Ela faz)

- No GitHub, cria PRs para cada branch (base: main). - Revê, comenta (Tu podes dar feedback). - Merge e apaga branches remotas. - Ambos atualizam local:

```
git checkout main
git pull origin main
```

Estrutura final na main:

```
projeto-web-simples/
  README.md
  index.html
  styles.css
  script.js
```

- Testa com Live Server no VS Code.

4.5 Passo 5: Limpeza de Branches Locais

Após merge, apaga branches locais:

```
git fetch --prune % Atualiza estado remoto
git branch -d feature/html-basico % Repete para outras
```

5 Resolvendo Conflitos

Se edits colidirem:

```
git checkout main
git merge feature/nome-da-branch % Mostra conflito
```

- Edita o ficheiro (remove « e »). - git add . e git commit.

6 Comandos Úteis

- git status: Mostra estado atual.
- git log: Histórico de commits.
- git branch: Lista branches.
- git fetch origin: Atualiza info remota.
- git rebase main: Limpa histórico na branch.

7 Boas Práticas

- Usa nomes descritivos: feature/nome, bugfix/nome.
- Commits pequenos com mensagens claras.
- Revê PRs antes de merge.

- Cria `.gitignore` para ignorar ficheiros (ex: `"node_modules/"`).
- Sempre `git pull` antes de push.
- Limpa branches após merge.

8 Dicas Extras

- Usa VS Code para editar e testar (Live Server para preview web).
- Para ajuda: `git --help` ou pesquisa online.
- Pratica com projetos como este use case para ganhar confiança!